



SOLVING THE LORENZ ODE SYSTEM USING OPTIMAL ANN ARCHITECTURES

Ikramul Hasan Moral • Md. Abu Bakar • Samiur Rahman Omlan • Fariha Islam • Md. Touhidul Islam

Department of Computer Science and Engineering, United International University

Team Name: Team_ParaDox • **Supervisor:** Dr. Muhammad Nomani Kabir, Professor, Dept. of CSE • **Course Teacher:** Dr. Md. Motaharul Islam, Professor, Dept. of CSE

Introduction & Motivation

Coupled nonlinear ODEs like the **Lorenz-1960** system — an early meteorological model of three coupled equations — rarely have closed-form solutions. Classical solvers (Runge-Kutta) are accurate but **mesh-dependent** and re-solve step-by-step for every query. A trained network can instead learn the continuous map $t \mapsto (x, y, z)$, yet most neural ODE work relies on physics-informed losses or operator learning.

Central question

Can a **plain feedforward ANN** — trained only on numerical data, no physics in the loss — approximate the Lorenz-1960 solution, and *which architecture does it best*?

Objectives

- Generate a trusted reference with RK4 *and* SciPy; cross-validate the two.
- Systematically vary **depth**, **width**, and **activation** (plain ANN).
- Evaluate accuracy vs. cost: MSE, MAE, max error, training/inference time.
- Benchmark the best plain ANN against PINN / DeepONet results.

Research Questions & Gap

Gap. No prior work runs a systematic plain-ANN architecture study on Lorenz-1960; existing studies fix the architecture, use only simple ODEs, or rely on physics-informed losses.

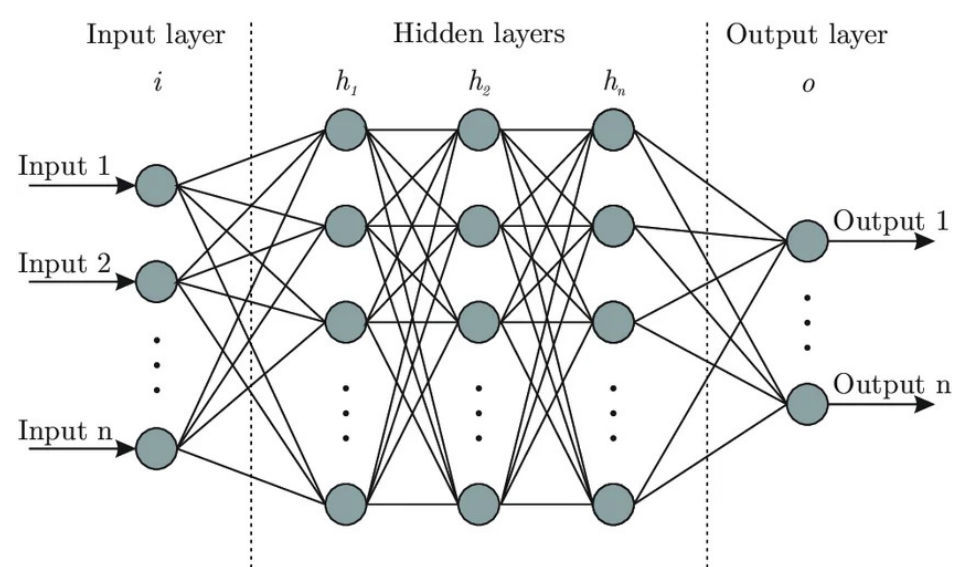
- Can a plain ANN learn the solution from numerical data alone?
- Which depth $\times$ width $\times$ activation gives the best accuracy–cost trade-off?
- How close does it come to physics-informed baselines?

Background: System & Network

Three coupled nonlinear ODEs; with $k = 2$, $l = 1$:

$$\begin{aligned}\dot{x} &= kl \left(\frac{1}{k^2 + l^2} - \frac{1}{k^2} \right) yz \\ \dot{y} &= kl \left(\frac{1}{l^2} - \frac{1}{k^2 + l^2} \right) xz \\ \dot{z} &= \frac{kl^2}{2} \left(\frac{1}{k^2} - \frac{1}{l^2} \right) xy\end{aligned}$$

$x_0=0.5$, $y_0=0.75$, $z_0=1$, $t \in [0, 1]$.



Plain ANN: learns $t \mapsto (x, y, z)$ by backpropagation. PINNs are used only as a literature benchmark.

Complex Engineering Problem

Mapped to the CEP rubric — **problem solving** (depth of knowledge, conflicting requirements, depth of analysis, interdependence), **activities** (resources, interaction, innovation, unfamiliarity), and **knowledge** (natural sciences, mathematics, engineering fundamentals, specialist ML, research literature, ethics).

Methodology: Two-Phase Study

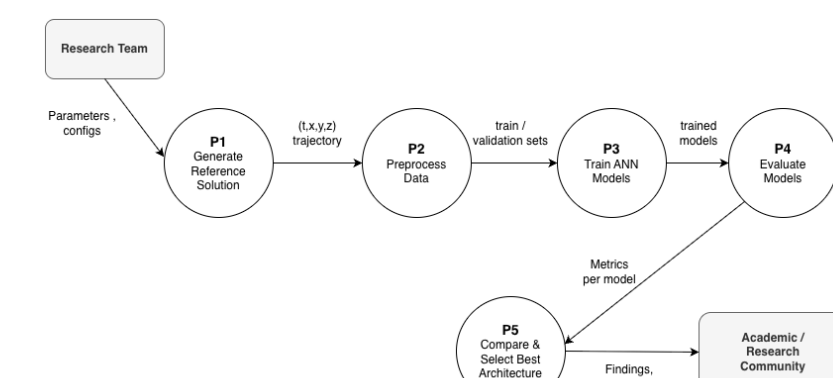
Phase 1 — Architecture search

depth $\{1, 2, 3, 4\} \times$ width $\{20, 50, 100\} \times$ activation $\{\tanh, \text{ReLU}, \text{sigmoid}, \text{GELU}, \text{Swish}\}$
= **60 runs** (Adam fixed)

Phase 2 — Optimizers

3 best architectures $\times \{\text{Adam}, \text{L-BFGS}, \text{SGD+momentum}\}$
= **9 runs**

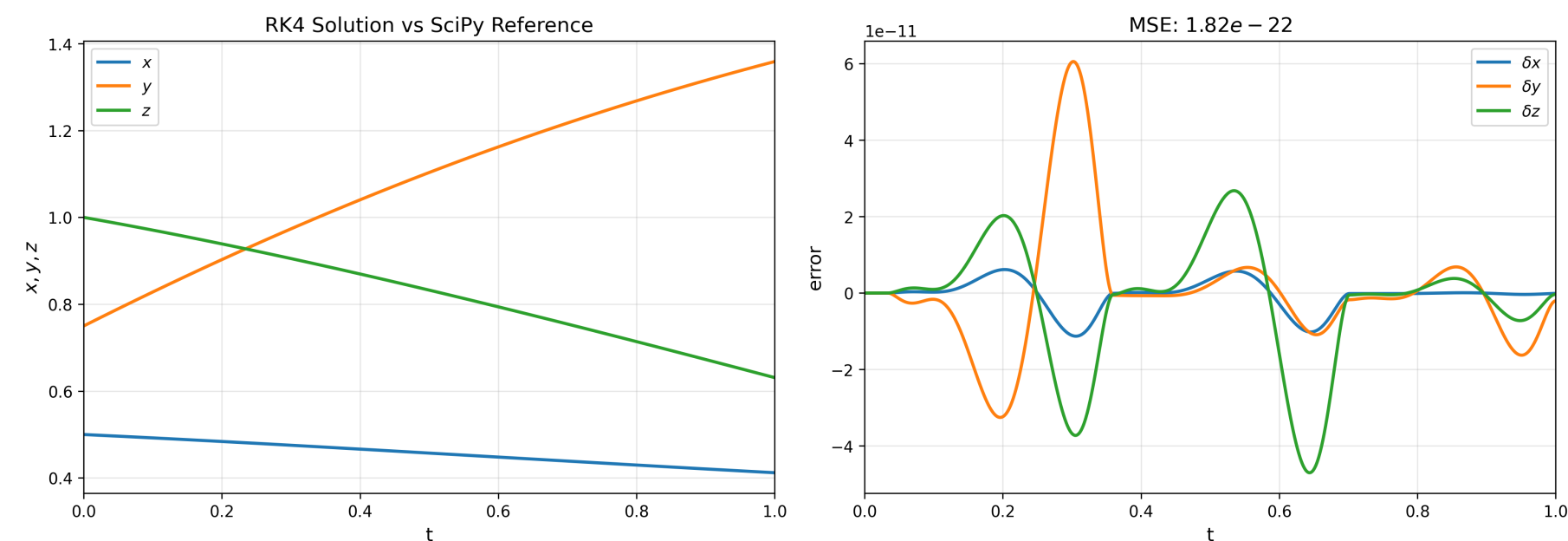
69 controlled experiments — this split cleanly separates architecture effects from optimizer effects.



Pipeline: generate $\rightarrow$ preprocess $\rightarrow$ train $\rightarrow$ evaluate $\rightarrow$ select.

Numerical Baseline & Validation (Result)

The reference trajectory is generated by two independent solvers and cross-checked before any ANN sees it: a custom **RK4** integrator ($h=10^{-3}$, 1001 points) and **SciPy DOP853** ($\text{rtol}=10^{-10}$), with an 80/20 train/validation split (seed 42).



The two solvers agree to **MSE** $\approx 1.8 \times 10^{-22}$, far below the 10^{-10} target — a defensible **ground truth** for all FYDP-II ANN training.

Conclusion & Future Work

Status (FYDP-I): research question framed, 17-paper review with an explicit gap, two-phase design (69 runs) locked, dual-solver baseline verified. No accuracy claims are made until the FYDP-II experiments produce evidence.

- Build the ANN training pipeline; run Phase 1 (60), then Phase 2 (9).
- Repeated seeds for stability; denser grids and new initial conditions.
- Direct comparison with PINN / operator-learning baselines.

Selected References

- [1] Lorenz (1960), *Tellus* 12(3). [2] Matthews & Bihlo (2024), PinnDE.
[3] Raissi et al. (2019), *J. Comput. Phys.* 378. [4] Lagaris et al. (1998), *IEEE TNN* 9(5).
[5] Lau & Werth (2020), ODEN. [6] Mall & Chakraverty (2013). [7] Chen et al. (2018), Neural ODEs.